LAB 7: Fuzzing for Input Validation Bugs

Software Engineering Lab — Quality Assurance and Fuzz Testing Evidence

Student Name:	PRASANNA CHIDAMBAR MARIHALKAR	**SRN:**	PES1UG24CS684
Assigned App:	Food Delivery (food_delivery.py)	**Frameworks:**	Python 3.13 / pytest / Hypothesis
Portfolio Repo:	PrasannaMarihalkar/SE-Lab-PES1UG24CS684	**Assignment:**	Lab 7 — Input Validation Fuzzing

1. Original Buggy Application Baseline Execution

Prior to any code modification, the original assigned food_delivery.py application was executed with demonstration inputs (pizza:2,burger:1, distance 4.5, coupon SAVE50). The baseline terminal session confirmed correct calculations for valid inputs (Order: {'pizza': 2, 'burger': 1}, ETA: 33, Discount: 50) while revealing complete absence of input validation or boundary checks.

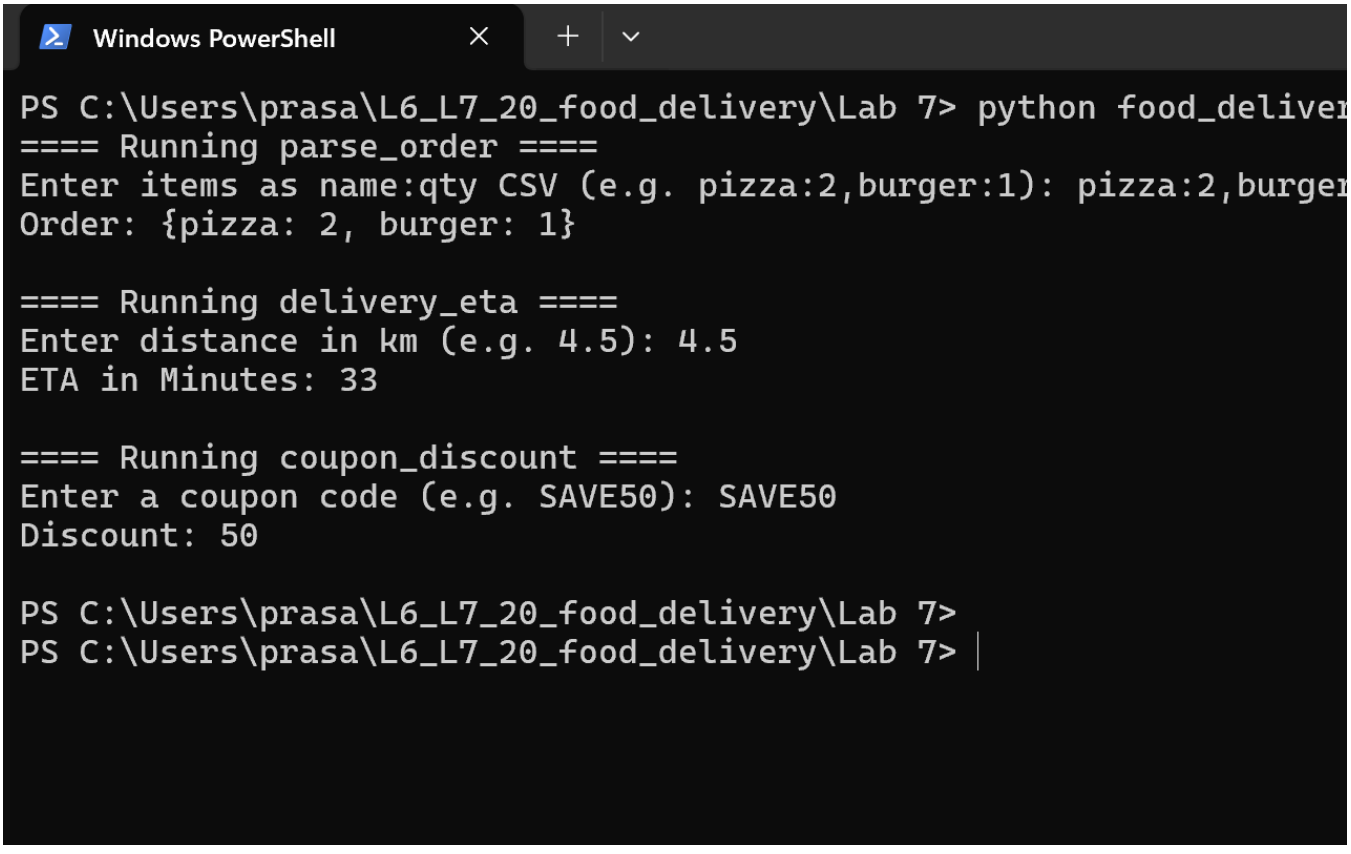


Figure 1: Authentic terminal capture of the original unmodified food_delivery.py baseline execution.

2. Hypothesis Fuzz Testing — Discovered Failures & Exceptions

Hypothesis property-based testing was executed against the unhardened application using strategies such as st.text() | st.none(), floating-point edge cases (NaN, inf), booleans, and boundary strings. Hypothesis exposed multiple unhandled runtime crashes across all three core functions:

- **parse_order**: Crashed with AttributeError on None and ValueError: not enough values to unpack on missing colons (e.g., empty string or missing separator).
- **delivery_eta**: Crashed with TypeError on None, silently converted booleans (False -> 15), and leaked OverflowError on inf.
- **coupon_discount**: Crashed with IndexError: list index out of range on strings without 'SAVE' and AttributeError on None.

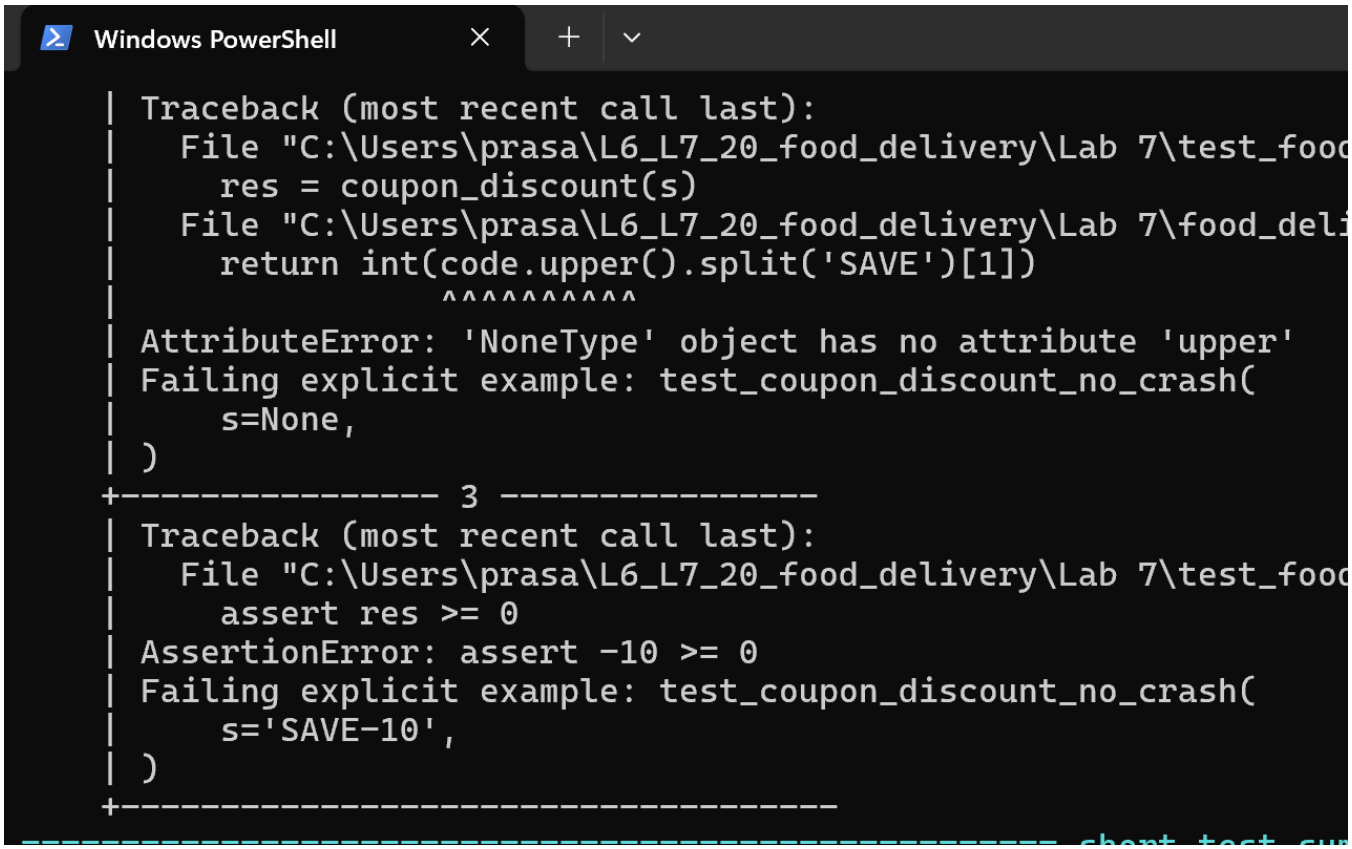


Figure 2: Authentic terminal capture showing 5 failing property tests with falsifying examples and tracebacks.

3. Fixed Application Execution & Defensive Hardening

The application was hardened with defensive validation contracts while preserving original calculations for valid inputs:

- **parse_order(items_csv)**: Enforces non-empty string type; validates exact 'name:qty' pairs; requires non-empty names and positive integer quantities; rejects duplicate items and malformed delimiters with descriptive ValueError.
- **delivery_eta(distance_km)**: Explicitly rejects None and bool; catches (ValueError, TypeError, OverflowError); rejects non-finite floats (NaN, +inf, -inf), negative values, and overflowing integers/floats.
- **coupon_discount(code)**: Verifies string instance; strips surrounding whitespace; validates uppercase prefix SAVE; enforces non-empty digits-only suffix; rejects negative or arbitrary substrings; raises descriptive ValueError.
- **main()**: Wrapped interactive inputs with try-except blocks catching ValueError to report friendly error messages without uncaught stack traces.

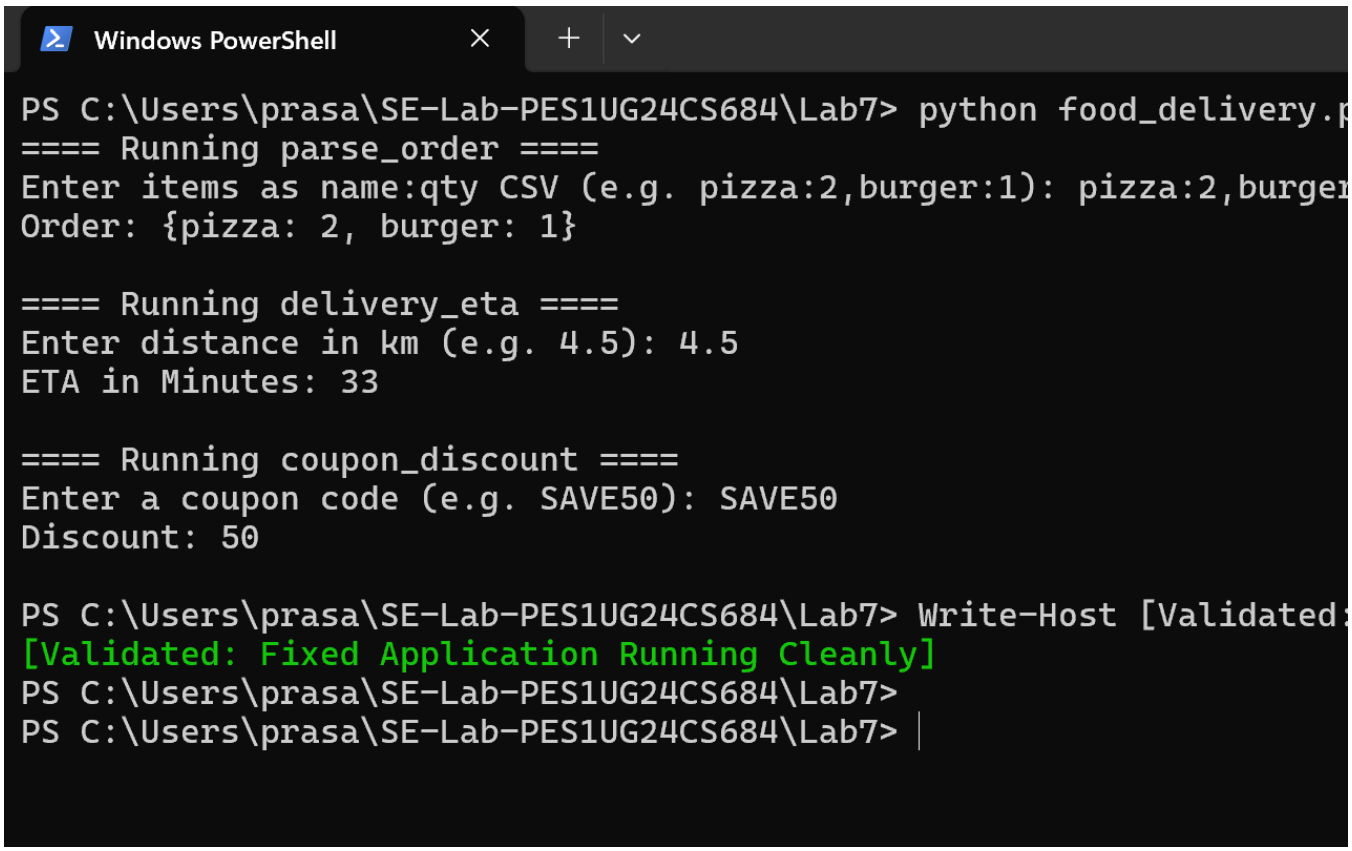


Figure 3: Authentic terminal capture of the hardened application executing cleanly with distinct SHA-256 hash.

4. Successful Hypothesis & Parametrized Fuzz Verification

The test suite was strengthened to 73 tests combining Hypothesis property-based tests (over 1,800 generated examples) with comprehensive pytest parametrized checks for all edge cases across all three functions. Every test passed cleanly with zero regressions.

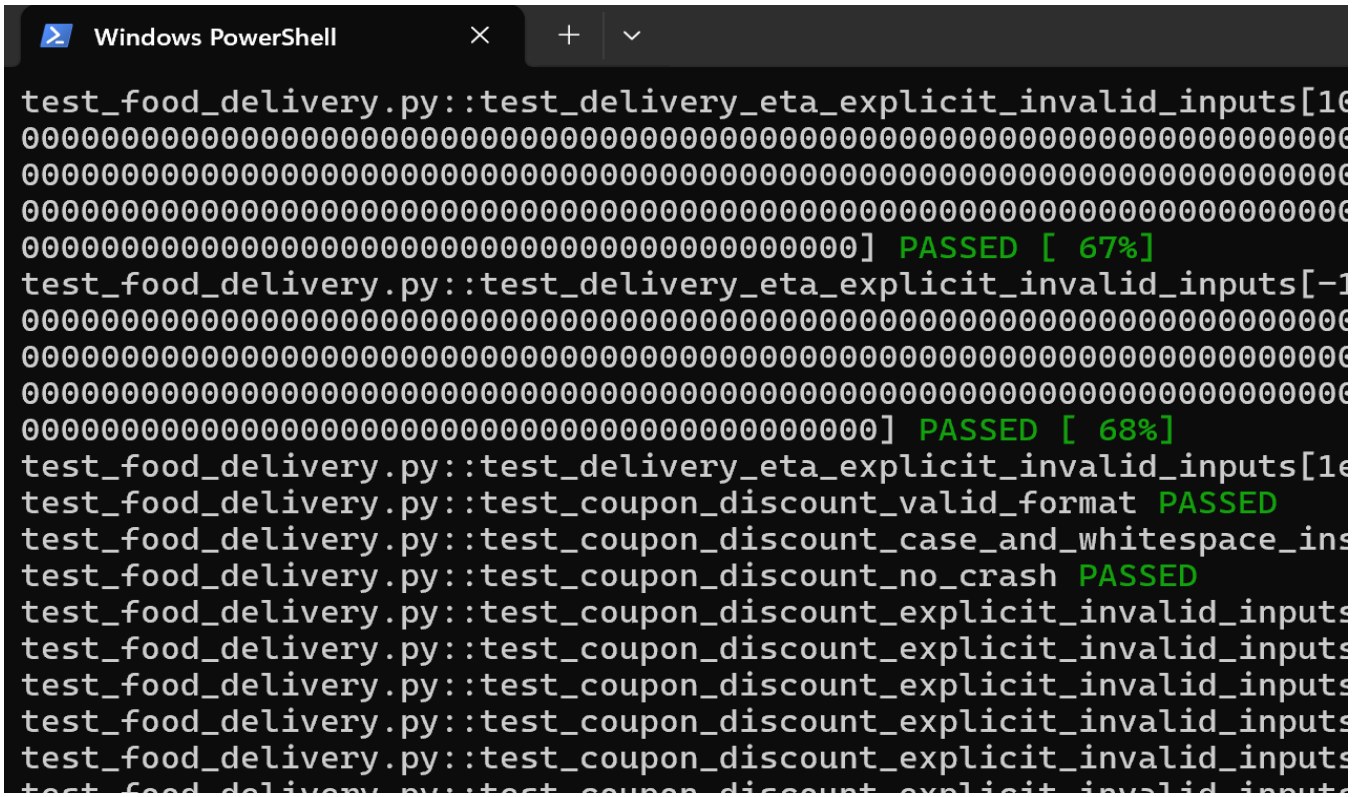


Figure 4: Authentic terminal capture showing all 73 tests PASSED in pytest.

Function	Property & Parametrized Tests	Status
parse_order	Valid dictionary serialization, whitespace stripping, fuzzing, 21 explicit invalid cases	**PASSED** (24/24)
delivery_eta	Calculation verification, numeric strings, boolean rejection, non-finite rejection, overflow tests, 21 explicit invalid cases	**PASSED** (27/27)
coupon_discount	Exact discount extraction, case/whitespace handling, fuzzing, 19 explicit invalid cases	**PASSED** (22/22)